

SmolVLM

Redefining Small and Efficient Multimodal Models

Hugging Face & Stanford • compact VLMs for mobile and edge inference

0.8 GB

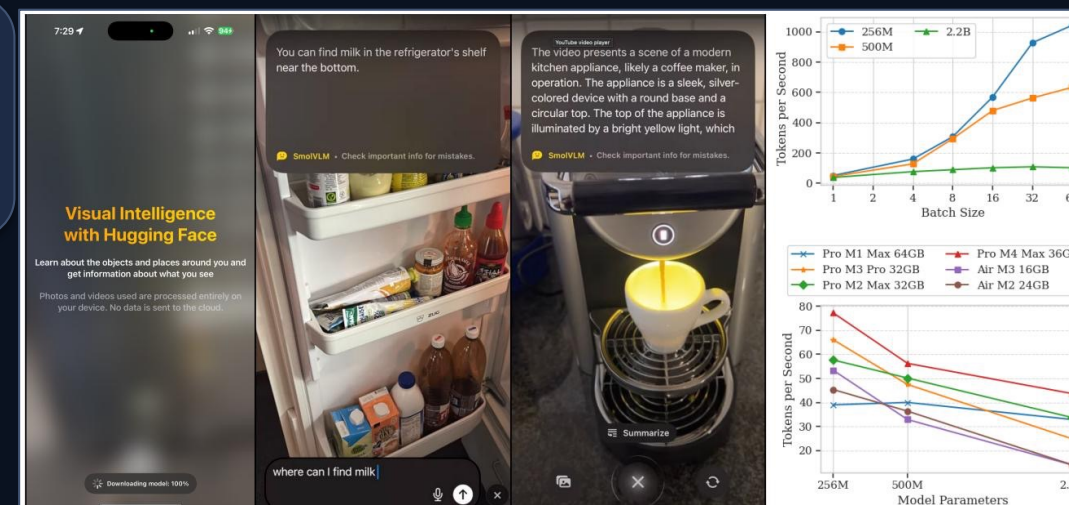
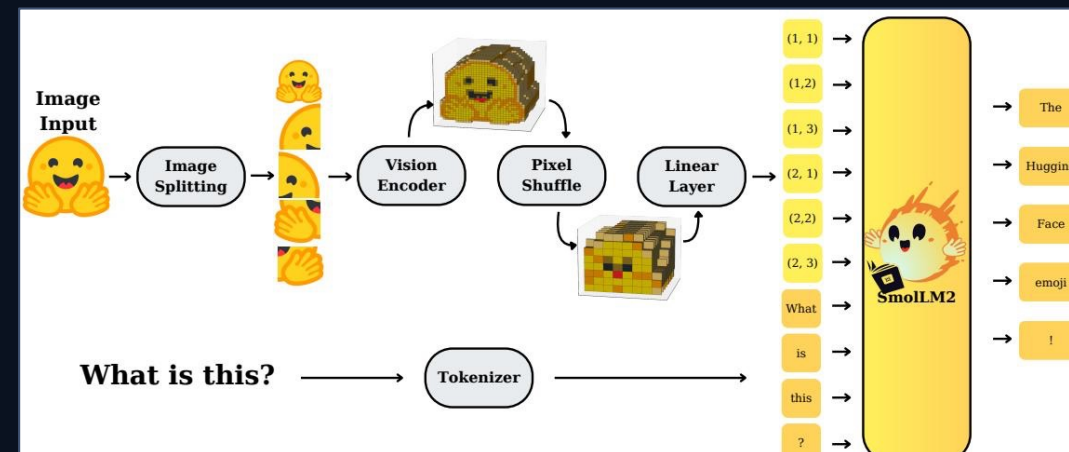
SmolVLM-256M RAM at
batch=1

59.8%

2.2B average across listed
benchmarks

~80 tok/s

256M on Pro M1 Max 64GB



Large VLMs Are Powerful but Impractical for Edge Deployment

Edge constraint

Mobile and low-power edge devices cannot absorb large VLM memory and compute demands without sacrificing latency, privacy, or battery life.

Memory pressure

Molmo-A1B-7B requires 27.7 GB RAM in the source comparison. SmolVLM-256M runs at 0.8 GB; SmolVLM-2.2B at 4.9 GB.

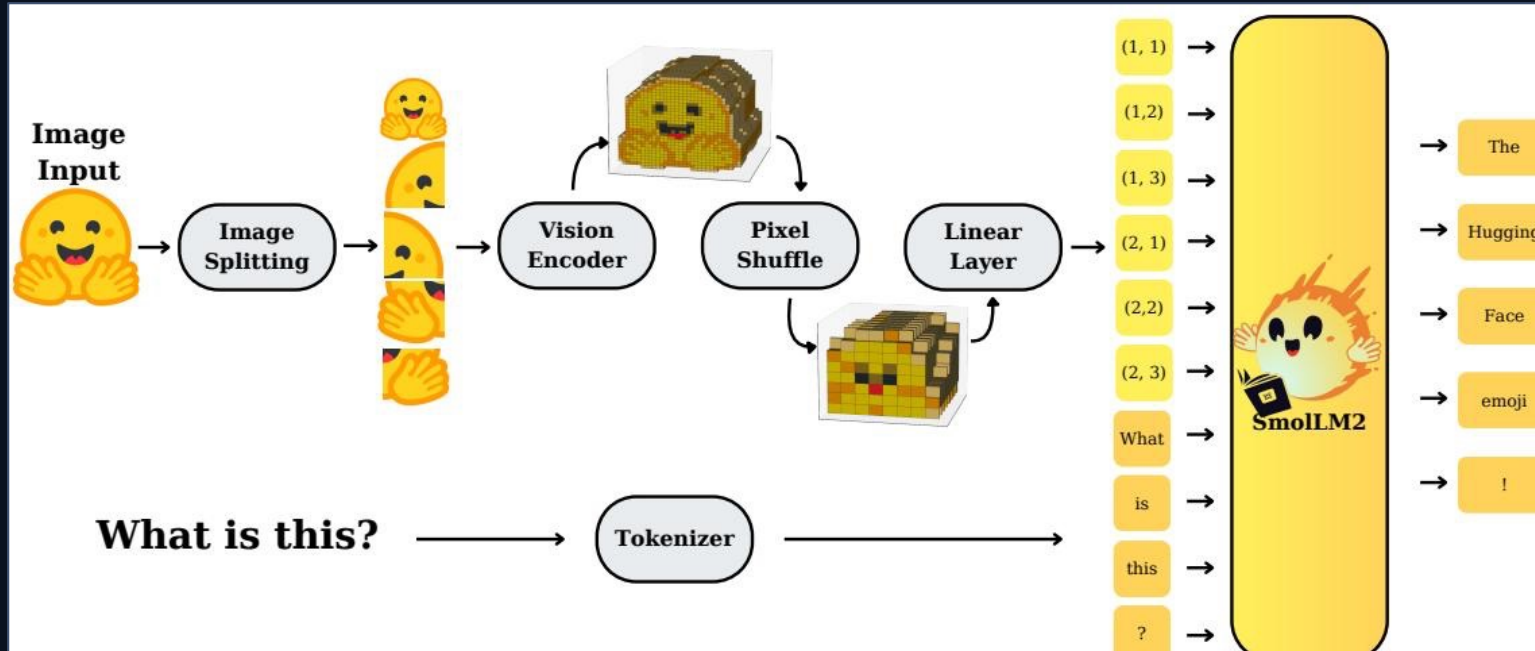
Efficiency result

The smallest SmolVLM outperforms the 300× larger Ldefics-80B, showing architectural efficiency can dominate raw scale.

RAM footprint at batch=1



SmolVLM Architecture: Image Splitting, Pixel Shuffle, and SmolLM2 Backbone



1. Split images

Images are split into sub-images; video frames are sampled and processed similarly.

2. Encode vision

A SigLIP vision encoder extracts visual features.

3. Compress tokens

Pixel shuffle rearranges spatial features into channels.

4. Project + fuse

A linear layer maps features into SmolLM2 input space, then text tokens are concatenated.

Smaller Vision Encoders Outperform Larger Ones in Compact VLMs

Same 135M language model; vision encoder size changes

SigLIP-B/16

93M vision parameters



OUTPERFORMS

paired with 135M LM

SigLIP-SO400M

428M vision parameters



UNDERPERFORMS

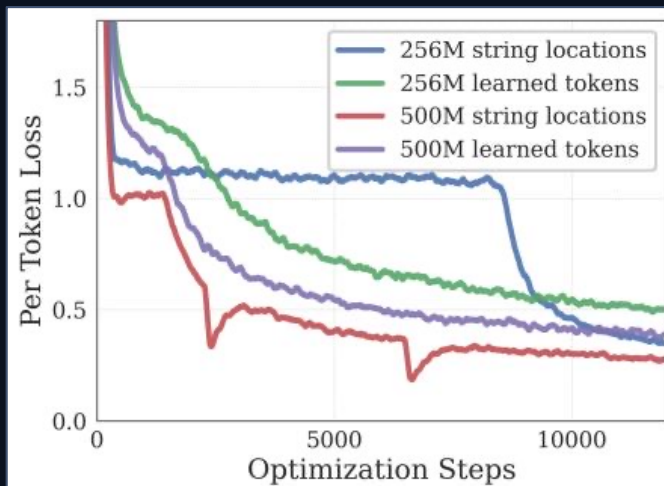
same LM; 4.6^S vision params

Design implication

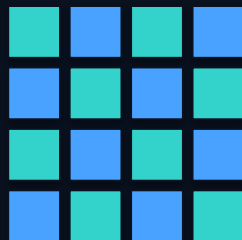
For compact VLMs, do not spend a disproportionate share of parameters on vision encoding. The larger encoder becomes justified only at 1.7B LM scale, where it adds 10% parameter overhead.

Balanced encoder-LM compute beats “bigger vision encoder” by default at small LM scale.

Aggressive Pixel Shuffle (r=4) Cuts Visual Tokens 16× for Small Models



Space-to-depth token compression



16 spatial tokens

$r = 4 \Rightarrow r^2 = 16\times$ fewer visual tokens

1 token
×16
channels

Small models

Benefit from $r=4$ compression:
lower attention overhead and better
long-context modeling.

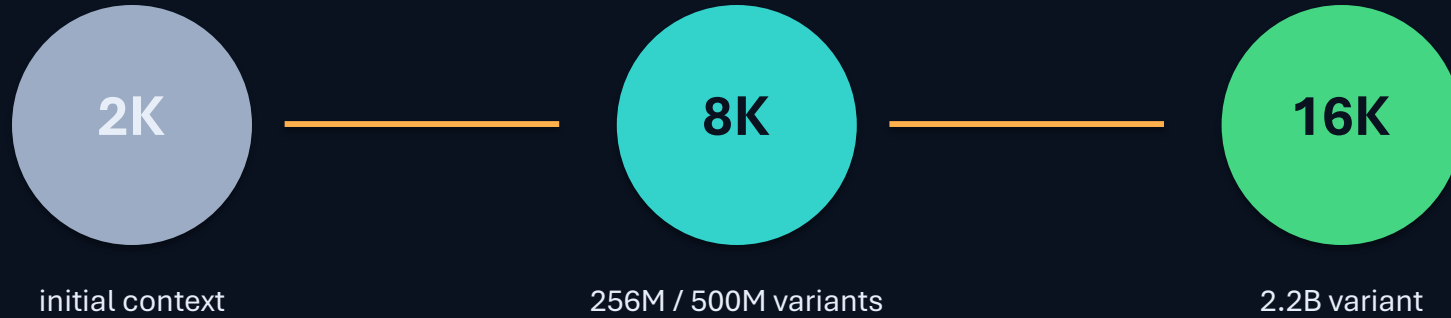
Larger models

Use $r=2$ in the source design.

Mechanism

Rearranges spatial features into
channels; no extra visual tokens are
kept.

Extending Context from 2K to 16K Tokens Unlocks Higher Image Resolution



RoPE scaling

Increase RoPE base from 10K to 273K.

Fine-tuning mix

Train on mixed long- and short-context data.

Qualitative impact

Longer context enables higher image resolution and improves performance.

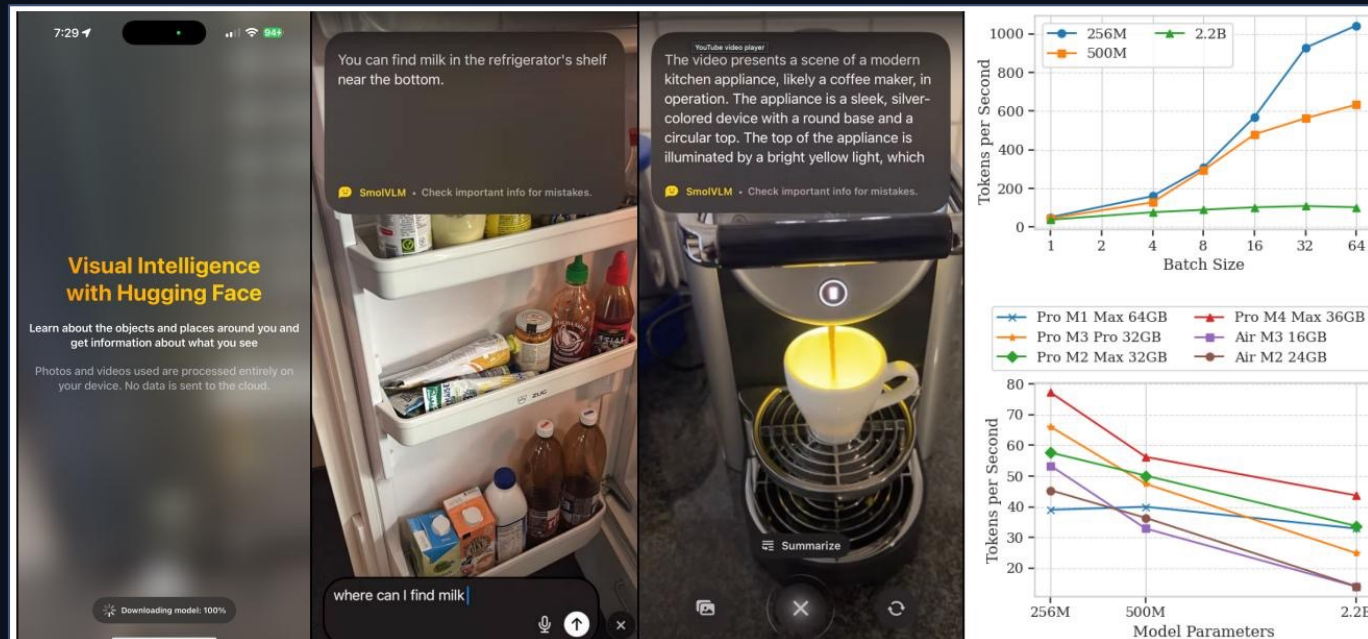
SmolVLM Achieves Competitive Performance at Sub-1GB to 4.9GB RAM

Capability	Benchmark	256M	500M	2.2B	Best Efficient OS
Single	OCRBench	52.6%	61.0%	72.9%	54.7% Molmo-A1B-7B
Single	AI2D	46.4%	59.2%	70.0%	71.0% Molmo-A1B-7B
Single	ChartQA	55.6%	62.8%	68.7%	48.0% Molmo-A1B-7B
Single	TextVQA	50.2%	60.2%	73.0%	61.5% Molmo-A1B-7B
Single	DocVQA	58.3%	70.5%	80.0%	77.7% Molmo-A1B-7B
Single	ScienceQA	73.8%	80.0%	89.6%	87.5% Molmo-A1B-7B
Multi	MMMU	29.0%	33.7%	42.0%	33.9% Molmo-A1B-7B
Multi	MathVista	35.9%	40.1%	51.5%	37.6% Molmo-A1B-7B
Multi	MMStar	34.6%	38.3%	46.0%	43.1% Molmo-A1B-7B
Video	Video-MME	33.7%	42.2%	52.1%	45.0% InternVL2-2B
Video	MLVU	40.6%	47.3%	55.2%	48.2% InternVL2-2B
Average	All listed	44.0%	51.0%	59.8%	—
RAM	Batch=1	0.8 GB	1.2 GB	4.9 GB	27.7 GB Molmo-A1B-7B

Key paper finding: SmolVLM-256M surpasses the 300× larger Idefics-80B.

Efficient OS baseline is Molmo-A1B-7B for image/multitask rows and InternVL2-2B for video rows.

Real-Time Inference on iPhones: Up to 80 Tokens/Second on Apple Silicon



~80 tok/s

SmolVLM-256M on Pro M1
Max 64GB

~55 tok/s

SmolVLM-256M on
MacBook Air M3 16GB

~1000

tok/s
256M at batch size 64 on
high-end devices

Scaling

Throughput scales
roughly linearly with
batch size for the
smallest model.

- HuggingSnap processes photos and videos entirely locally.
- No cloud dependency in the mobile app deployment path.
- Sub-1GB RAM puts multimodal inference within smartphone and edge budgets.

Key Takeaways: Design Principles for Efficient Multimodal Models

Balance compute

Match vision encoder size to LM scale; smaller encoders can win for compact LMs.

Scale context deliberately

Extend context with RoPE-base adjustment and mixed long/short-context fine-tuning.

Compress visual tokens

Use aggressive pixel shuffle $r=4$ for small VLMs to reduce tokens by 16 $\times$.

Target memory first

Sub-1GB inference is achievable: 0.8 GB for SmolVLM-256M at batch=1.

Open the stack

Weights, datasets, code, and HuggingSnap are fully open-sourced for reproducibility.

Implication: efficient multimodal design is a systems problem — allocate parameters, tokens, context, and memory together.